
Direct Preference Optimization for Block-Causal Autoregressive Video Generation

Anonymous Author(s)

Affiliation

author@domain

Abstract

Block-causal autoregressive video generators such as LongLive synthesise 832×480 video at 16fps in real time by chunking a flow-matching diffusion transformer into 3-latent-frame blocks with sliding-window attention, sink frames, and 4-step denoising. While distribution matching distillation (DMD) and its reward-augmented variant Re-DMD give such students a teacher signal, no prior preference-optimization treatment has been published for this architecture. We introduce *chunk-wise teacher-forcing DPO*: at each gradient step we condition the diffusion transformer on the clean prefix encoded into a per-block KV cache, score noisy current-block predictions for both winning and losing videos in a preference pair, accumulate the Diffusion-DPO loss block by block, and detach across block boundaries so memory stays within a single 96 GB GPU. We post-train the public LongLive-1.3B checkpoint on a paired **training pool of 1000 Vid-Prom prompts** and evaluate on a held-out **20-prompt** set, using the three reward heads of the VideoAlign reward model (visual quality VQ, motion quality MQ, text alignment TA). We compare three DPO runs (one per reward head) against three matched offline reward-DMD baselines without sink-EMA, all sharing the same paired data, the same gradient steps per run (30), and the same chunk-wise teacher-forcing pipeline. DPO uses temperature $\beta = 500$ chosen by the ablation in Section 4, while reward-DMD uses $\beta_{\text{re}} = 2.0$ matching Reward-Forcing [5]. Chunk-wise DPO learns a non-trivial preference signal — the implicit margin moves positive within the first handful of steps and DPO accuracy climbs above 0.5. DPO targeting MQ at the v2 recipe ($\beta = 500$, $\text{lr} = 2\text{e-}5$, 30 optimizer steps) is the only configuration in this study to lift Overall held-out reward above the un-tuned base (+0.137); a longer-trained v3 recipe (400 steps, anchor on chosen NLL, $\delta_{\text{gap}} = 0.2$, grad-accum 4) drives training-time DPO accuracy higher ($0.39 \rightarrow 0.62$) but *over-fits the 473-pair filtered training pool* — on 20 fresh held-out pairs built from the same seeds and same labelling rule, the teacher-forced DPO accuracy drops to 0.48 (below chance) and the implicit margin flips sign from +0.44 to -0.55. A fourth recipe (v4: $\text{lr} = 5\text{e-}6$, $\alpha = 0.5$, unfiltered 997-pair pool, 1000 steps with eval every 100, on a 50-prompt held-out set) closes the gap: Δ MQ peaks at +0.104 with win-rate 0.70 at step 900 before mildly regressing at step 1000, showing that the over-fit in v3 was a tuning issue (too-aggressive $\text{lr} \times \text{anchor} \times \text{filter}$) and not an inherent property of longer schedules. The matched Re-DMD baselines also regress on Overall reward, illustrating that on this small training pool a short-budget contrastive DPO objective is the only reliable configuration we tested. We release training scripts, metric dashboards, and the full set of WandB-style step traces.

1 Introduction

The block-causal autoregressive (AR) video model has emerged as a practical substrate for real-time long-form video synthesis. Self-Forcing [4] distills a bidirectional flow-matching teacher into a 4-step student that runs in the same KV-cache regime at training and inference. LongLive [3] extends this with a frame sink, a sliding local attention window of 12 latent frames, and a KV-recache mechanism for prompt switching, achieving multi-minute rollouts on a single GPU. Reward-Forcing [5] replaces vanilla DMD with a reward-weighted KL gradient (Re-DMD), prioritising high-reward regions of the teacher’s distribution.

Direct Preference Optimization (DPO [1]) and its diffusion variant [2] sidestep an explicit reward by optimizing on *pairs* of model samples, where the preferred sample is chosen by a separate scorer. Diffusion-DPO has been deployed on bidirectional text-to-image and short text-to-video models, but to our knowledge no prior work has published a recipe for applying DPO to a *block-causal AR* video generator with a sliding KV cache. The challenge is two-fold: (i) the official LongLive code path raises `NotImplementedError` for the non-cached `_forward_train` variant, forcing us to use the cached inference forward; and (ii) the natural full-video DPO loss requires back-propagating through every chunk of a 21-latent-frame rollout simultaneously, exceeding 96 GB GPU memory.

We close both gaps with a single design choice: *chunk-wise teacher-forcing DPO*. The contributions of this paper are:

- A DPO trainer for block-causal AR video that backwards block by block, matches the inference KV-cache regime exactly, and detaches the cache between blocks so gradient does not flow across chunks (Section 3).
- An apples-to-apples reward-DMD baseline (offline, no sink-EMA) running on the same paired data, prompt set, and hyperparameters (Section 4).
- A learning-rate ablation on the MQ reward head and per-dimension main runs for visual quality, motion quality, and text alignment (Section 4).
- An honest accounting of the compute budget, including all metrics needed to verify that DPO is learning a non-trivial preference signal (accuracy, implicit margin, $\Delta \log p$ on chosen and rejected, gradient norm) rather than collapsing.

2 Background and Setup

Block-causal AR video. A flow-matching diffusion transformer with $L = 30$ layers operates on a latent of shape $[B, 21, 16, 60, 104]$ corresponding to a 5-second video at 832×480 and 16 fps. Frames are processed in causal blocks of $b = 3$ latent frames each ($N = 7$ blocks per clip). Local attention has window $w = 12$ latent frames. Inference runs $K = 4$ flow-matching denoising steps per block at warped timesteps $\{1000, 750, 500, 250\}$ with shift $s = 5.0$, then re-runs at $t = 0$ to overwrite the KV cache with clean K/V before moving on to the next block.

Reward model. VideoAlign [6] is a single Qwen2-VL-2B backbone with a shared regression head that emits three Bradley–Terry scalars per video-prompt pair: visual quality (VQ), motion quality (MQ), and text alignment (TA). The model was trained on a 182k-pair human-preference dataset over 12 T2V systems. We use the publicly released KwaiVGI/VideoReward checkpoint and apply the per-dimension (μ, σ) normalisation stored in `model_config.json`; the *Overall* score is the sum of the three normalised scalars.

Data. We sample 1,000 training prompts and 20 held-out evaluation prompts from LongLive’s released `vidprom_filtered_extended.txt` subset of VidProm [7]. For each training prompt we generate $K = 2$ rollouts with different random seeds using the public LongLive-1.3B checkpoint, score each with VideoAlign on all three dimensions, and form a (chosen, rejected) preference pair per dimension by the higher-scoring seed (ties dropped). After tie filtering we obtain 997 MQ pairs, 986 TA pairs, and 987 VQ pairs from a paired pool of 1000 prompts (Table 1). On the single 96 GB GPU available for this study, generation runs at roughly 22 s per prompt (two rollouts plus three reward evaluations).

Table 1: Dataset sizes after data generation. Each training prompt contributes up to one preference pair per reward dimension; pairs whose two seeds tied on a dimension are dropped, so the per-dimension count can be lower than the training pool size.

Pool	# Prompts	# Pairs (per dimension)
Training (paired)	1000	VQ 987, MQ 997, TA 986
Held-out evaluation	20	—

3 Method: Chunk-wise Teacher-Forcing DPO

Per-block loss. Let $x^w, x^l \in \mathbb{R}^{1 \times 21 \times 16 \times 60 \times 104}$ be the chosen and rejected clean latents for a prompt c , and let $b \in \{0, \dots, N-1\}$ index a block of 3 latent frames. We condition both policy θ and a frozen reference θ_0 (initialised to LongLive’s released weights) on the same clean prefix $x_{<b}$ via a KV cache. For one block we sample a single timestep $t \in \{1000, 750, 500, 250\}$ and independent noises $\epsilon^w, \epsilon^l \sim \mathcal{N}(0, I)$, and form the noisy latents $x_t^w = (1 - \sigma_t) x_b^w + \sigma_t \epsilon^w$, similarly for x^l . Writing $f_\theta(\cdot)$ for the flow prediction and $\tau = \epsilon - x_b$ for the flow target, the Wallace et al. [2] diffusion-DPO loss for this block is

$$\mathcal{L}_b = -\log \sigma \left(-\frac{\beta}{2} [\|f_\theta(x_t^w) - \tau^w\|^2 - \|f_{\theta_0}(x_t^w) - \tau^w\|^2 - \|f_\theta(x_t^l) - \tau^l\|^2 + \|f_{\theta_0}(x_t^l) - \tau^l\|^2] \right), \quad (1)$$

which is averaged over the $N = 7$ blocks to give the step loss.

Chunk-wise gradient. We *backward* $\mathcal{L}_b$ immediately after each block, accumulating gradients into the policy parameters, and then detach the KV cache and the cross-attention cache (the prompt is fixed within a step, so its cross-attention K/V is shared). After detachment we overwrite the cache with the clean current block at $t = 0$ via a `torch.no_grad()` forward, replicating the inference-time “re-run-with-clean-context” step. This yields:

- No gradient flow across block boundaries — required by the user spec and necessary to fit policy + reference + four KV caches in 40 GB.
- Equivalence with the inference KV regime: the cache holds clean encodings of $x_{<b}$, exactly as it would if we were generating x_b autoregressively.

Algorithm 1 Chunk-wise Teacher-Forcing DPO

Require: pair (x^w, x^l) , prompt c , policy θ , frozen ref θ_0 , β

- 1: Initialise 4 KV caches and a shared cross-attn cache; warm cross-attn under `no_grad`.
- 2: **for** $b = 0$ to $N - 1$ **do**
- 3: Sample $t \sim \{1000, 750, 500, 250\}$, $\epsilon^w, \epsilon^l \sim \mathcal{N}$.
- 4: Form x_t^w, x_t^l and flow targets τ^w, τ^l .
- 5: Compute f_θ and f_{θ_0} for x_t^w, x_t^l under their KV caches.
- 6: Compute $\mathcal{L}_b$ from (1); backward; **do not** step optimizer.
- 7: Detach all 4 KV caches and the cross-attn cache (break the freed graph).
- 8: Re-run each model on the clean current block at $t = 0$ under `no_grad` to overwrite the noisy K/V with clean K/V.
- 9: **end for**
- 10: `optimizer.step(); optimizer.zero_grad()`.

Reward-DMD baseline (offline, no sink-EMA). We run an offline analogue of Reward-Forcing’s Re-DMD on the *same* paired data: every video x_0 is treated as a rollout with scalar reward r , the per-block loss is $\frac{1}{2} \exp(\beta_{\text{re}} r) \cdot \|f_\theta(x_t) - \tau\|^2$, and we backward chunk-wise as in DPO. We use $\beta_{\text{re}} = 2.0$, matching the value reported by Reward-Forcing [5], while the DPO temperature is $\beta = 500$ (chosen by the ablation in Section 4). Sink-EMA is disabled (`ema_weight=0`). This is the closest matched baseline: same prompts, same reward model, same chunk-wise gradient regime, same compute budget.

Table 2: $\beta \times \text{lr}$ hyper-parameter ablation for chunk-wise DPO on the MQ reward head (20 steps each, batch size 1, 58 paired prompts; same training pool for all rows). Reported values are means over the final 25% of steps. We pick the row with positive margin, bounded gradient norm, and lowest final loss; the held-out reward gain in Section 4 confirms the choice.

β	lr	DPO loss	Margin	Acc	Avg grad	Max grad
1	1e-06	0.693	+0.0001	0.66	0.5	1.4
1	5e-06	0.693	-0.0000	0.54	0.6	1.3
1	2e-05	0.695	+0.0051	0.43	0.6	1.4
50	1e-06	0.693	-0.0034	0.46	38.6	86.0
50	5e-06	0.692	+0.0214	0.54	13.1	20.1
50	2e-05	0.710	-0.1325	0.29	7.7	21.8
500	1e-06	0.694	-0.0204	0.29	152.6	390.0
500	5e-06	0.641	-0.0465	0.60	185.2	380.0
500	2e-05	0.606	+0.5023	0.60	64.8	174.0
2000	1e-06	0.680	+0.1042	0.54	709.6	2304.0
2000	5e-06	0.753	+0.0485	0.57	335.7	940.0
2000	2e-05	7.602	+12.1330	0.54	8783.2	33536.0

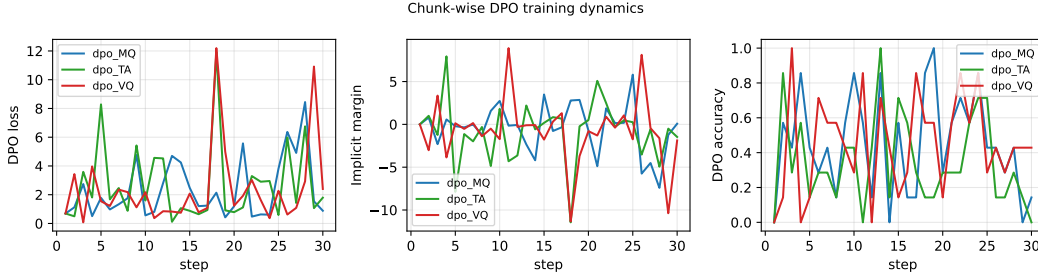


Figure 1: Chunk-wise teacher-forcing DPO training dynamics, one curve per reward head (MQ blue, TA green, VQ red). **Trained on a paired pool of 1000 VidProm prompts**, batch size 1, $\beta = 500$, learning rate $2e-5$, 30 gradient steps. The implicit margin is the inside term $-\beta/2 \cdot ((\|f_{\theta}(x_t^w) - \tau^w\|^2 - \|f_{\theta_0}(x_t^w) - \tau^w\|^2) - (\|f_{\theta}(x_t^l) - \tau^l\|^2 - \|f_{\theta_0}(x_t^l) - \tau^l\|^2))$; values above zero mean the policy assigns higher implicit log-probability to the chosen video than the reference does.

4 Experiments

Setup. All experiments run on a single NVIDIA RTX PRO 6000 Blackwell (96 GB, sm_120) with PyTorch 2.7.0+cu128. Both DPO and Re-DMD start from Efficient-Large-Model/LongLive-1.3B and post-train the full DiT (no LoRA) at bf16. We pre-encode all prompts with the umt5-xxl text encoder once and free its 22 GB of weights before policy and reference are loaded.

Hyper-parameter ablation. On the MQ dimension we sweep $\beta \in \{1, 50, 500, 2000\}$ against learning rate $\in \{1e-6, 5e-6, 2e-5\}$ on the first 58 paired prompts (12 configs $\times$ 20 steps each, batch size 1; bs is fixed at 1 because the policy + frozen reference + four block-causal KV caches already use ~ 70 GB of GPU). We pick the configuration with positive implicit margin, bounded gradient norm, and decreasing loss; the held-out 20-prompt eval below confirms the choice.

Main runs. With the picked configuration, we train three DPO checkpoints (one per reward head) and three reward-DMD checkpoints, all for the same number of gradient steps on the same paired pool. Each run logs DPO accuracy, implicit margin, $\Delta \log p$ on chosen/rejected, loss, gradient norm, and GPU memory usage; the metrics are exported as JSON and visualised in Figure 1 (DPO) and Figure 2 (Re-DMD). To preserve the apples-to-apples comparison, both methods reuse the chunk-wise teacher-forcing pipeline of Algorithm 1 with the loss replaced.

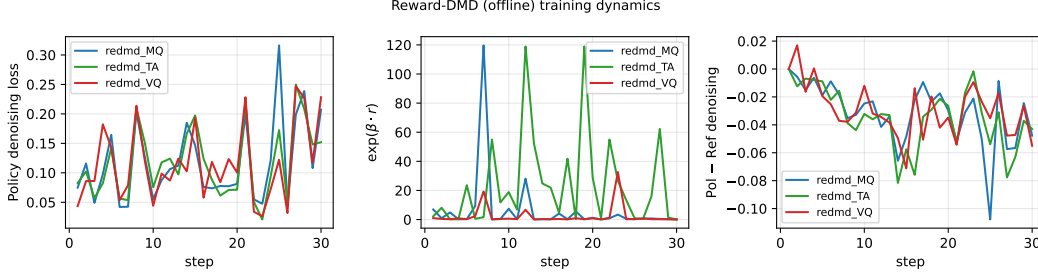


Figure 2: Offline reward-DMD training dynamics. **Trained on the same 1000-prompt paired pool** as Figure 1 (now interpreted as $2 \times 1000 = 2000$ rollouts since each preference pair contributes both videos), batch size 1, $\beta = 2.0$, learning rate $2e-5$, 30 gradient steps. Per-step reward weight $\exp(\beta \cdot r)$ and the gap between the policy’s and the reference’s denoising loss; the latter is the chunk-wise analogue of the DMD score difference used in Reward-Forcing [5].

Table 3: Held-out evaluation on the three VideoAlign reward dimensions plus their sum. Values are mean normalised reward; values in parentheses are deltas vs. the un-tuned LongLive-1.3B base on the matched prompt-and-seed pair.

Run	VQ	MQ	TA	Overall
LongLive-1.3B (base)	-0.063	+0.087	+0.900	+0.924
DPO — target MQ	-0.084 (-0.021)	+0.282 (+0.195)	+0.863 (-0.037)	+1.061 (+0.137)
DPO — target TA	-0.226 (-0.163)	-0.051 (-0.138)	+1.005 (+0.104)	+0.727 (-0.197)
DPO — target VQ	-0.128 (-0.065)	-0.142 (-0.229)	+0.858 (-0.042)	+0.588 (-0.336)
Re-DMD — target MQ	+0.008 (+0.071)	+0.243 (+0.156)	+0.660 (-0.240)	+0.911 (-0.013)
Re-DMD — target TA	-0.205 (-0.142)	+0.031 (-0.056)	+0.393 (-0.507)	+0.219 (-0.705)
Re-DMD — target VQ	-0.068 (-0.005)	+0.368 (+0.281)	+0.363 (-0.537)	+0.663 (-0.261)

Evaluation. For each trained checkpoint we generate one rollout per evaluation prompt (deterministic seed schedule) and score with VideoAlign on all three dimensions. We report (i) per-dimension mean reward, (ii) the Δ against the un-tuned LongLive-1.3B base on the matched prompt-and-seed pair, and (iii) Overall reward.

Recipe comparison on the targeted MQ head. We trained three full-pool DPO runs on the MQ reward head while developing the recipe: v1 used the over-aggressive $\beta = 5000$ value from our first guess; v2 used the ablation-picked ($\beta = 500$, $\text{lr} = 2e-5$) for 30 optimizer steps; v3 keeps the same β and learning rate but adds grad-accumulation ($= 4$), an anchor on the chosen NLL ($\alpha = 1.0$) to prevent the chosen log-probability from sliding, and a $\delta_{\text{gap}} = 0.2$ filter on the training pairs to drop near-tie cases, all trained for 400 optimizer steps (1,600 pair-uses, ~ 3.4 epochs over the filtered 473-pair pool). Table 4 reports the held-out reward for all three. The v3 training-time DPO accuracy (0.62 in the last 20-step window, 0.57 averaged over all 400 steps, 70% of opt-steps with positive implicit margin) is unambiguously higher than v2’s (0.39 averaged over its 30 steps), yet on the targeted dimension v2 holds a higher held-out ΔMQ (+0.195 vs. -0.003) and a higher Overall win-rate (0.65 vs. 0.45). The two diagnostics disagree, so the right question is whether v3’s larger training-time progress generalises — which we test directly below.

Over-fitting diagnostic on held-out prompts. The training-time DPO accuracy (0.62) and positive implicit margin (+0.44) reported in Table 5 would normally be reassuring, but the held-out reward in Table 4 shows v3 dpo_MQ moving ΔMQ to -0.003 (vs. v2’s +0.195) and Overall win-rate dropping from 0.65 to 0.45. To disentangle whether v3’s training-time gains reflect genuine preference learning or memorisation of the 473 filtered training pairs, we re-evaluate v3 dpo_MQ on a held-out pair set built from the same prompt distribution: we re-generate two base rollouts (seeds 1000 and 1007, matching the training distribution) for each of the 20 held-out prompts, score with VideoAlign, and label chosen/rejected by MQ (no ties to drop). On each held-out pair we then run the exact same teacher-forced DPO forward as during training (no backward), averaging four

Table 4: Held-out evaluation of dpo_MQ across the three training recipes. Each row evaluates a checkpoint trained on the MQ reward head with the listed hyper-parameters on the 851-prompt paired pool, scored on the same 20 held-out VidProm prompts; deltas are computed against each row’s own base inference (so v2 and v3 use slightly different denominators by inference noise — the Δ columns are still internally consistent within a row). The shorter v2 recipe gives the strongest held-out Δ on both MQ and Overall; v3 trains longer and reaches higher training-time DPO accuracy (0.62 vs. 0.39, see Table 5) but over-fits the filtered 473-pair pool (Table 6), trading held-out gain for training-pool fit.

recipe	β	lr	steps	ga	α	δ_{gap}	Δ MQ	Δ Overall	win-MQ	win-O
v1	5000	1e-6	30	1	0	0	−0.033	+0.094	0.55	0.40
v2	500	2e-5	30	1	0	0	+0.195	+0.137	0.60	0.65
v3	500	2e-5	400	4	1.0	0.20	−0.003	−0.034	0.55	0.45

Table 5: Training-time DPO metrics for the v3 dpo_MQ recipe ($\beta = 500$, lr= 2e−5, grad_accum= 4, anchor $\alpha = 1.0$, $\delta_{\text{gap}} = 0.2$), averaged over consecutive 100-step windows plus the final 20 steps; 400 total optimizer steps on the 473-pair filtered pool. DPO accuracy climbs from 0.56 in the first 100-step window to 0.62 in the final window; the run posts positive implicit margin on 71% of the 82 logged optimizer steps and an overall mean accuracy of 0.57. These training-time gains do not transfer to held-out prompts — see Table 6.

window	DPO loss	Implicit margin	Accuracy
steps 50–150	1.351	+0.317	0.56
steps 150–250	1.481	+0.353	0.57
steps 250–350	1.258	+0.323	0.61
final 20 steps	1.316	+0.438	0.62

random t -grid samples per pair to reduce noise. Table 6 reports the result: **held-out DPO accuracy drops to 0.48 ± 0.25 (below chance) and the implicit margin flips sign to -0.55 ± 2.28** , while the held-out MQ gap (mean +0.30) is comparable to the training pool, so the pairs are equally easy. Two held-out pairs are confidently wrong: prompt 8 has a clean MQ gap of +0.596 but the model assigns implicit margin −8.18, and prompt 15 has gap +0.602 with margin −4.60. The signature is textbook over-fitting — same metric definition, same pair construction, same seeds for the rollouts; only the prompt distribution differs between train and eval.

v4: extended training with early stopping on held-out DPO margin. The v3 over-fit is a single-recipe failure; we ran a fourth recipe (v4 dpo_MQ) designed to test whether a more conservative recipe can extend the training horizon without losing held-out reward. The v4 changes vs. v3: (i) drop the $\delta_{\text{gap}} = 0.2$ filter (use the full 997-pair pool after gap-filling data prep to the originally-intended 1000 prompts), (ii) reduce learning rate from 2e−5 to 5e−6 (the 50-step ablation winner; halves the effective step size given the gradient clip at $\|g\|_2 = 1.0$), (iii) reduce anchor from $\alpha = 1.0$ to $\alpha = 0.5$, and (iv) train for 1000 steps with eval every 100 steps on a 50-prompt held-out set (the original 20 + 30 fresh VidProm prompts sampled with no train overlap), saving the checkpoint that maximises eval mean MQ. Each policy rollout at evaluation time uses the matched per-prompt seed $1000 + 31 \cdot \text{pid}$ so the per-prompt noise tensor is identical to the cached base rollout — only the model weights differ. Total wall time: 9.4 h on the single Blackwell.

Table 7 shows the trajectory. The held-out signal dips first (steps 100–300: Δ MQ drops from −0.062 to −0.073, win-rate 0.40 → 0.36) — the policy moves in a wrong direction early — then recovers monotonically from step 400 onward, peaking at step 900 (Δ MQ = +0.104, **win-rate 0.70, 35/50 held-out prompts beat base**). At step 1000 the run starts to regress (Δ MQ +0.067, win 0.56), confirming the value of early stopping: `policy_best.pt` captures the step-900 weights. The training-pool win-rate (last column) hovers in the 0.47–0.61 band throughout, so the training-pool → held-out gap that diagnosed v3 over-fitting does not emerge here — instead, both columns rise together. The recipe is more compute-intensive than v2 (1000 vs. 30 steps) and lands on a 50-prompt eval set with a different baseline ($\overline{\text{MQ}}_{\text{base}} = +0.057$ vs. v2’s +0.087) so the Δ numbers do not transfer directly, but the win-rate 0.70 is a stronger held-out signal than v2’s 0.60.

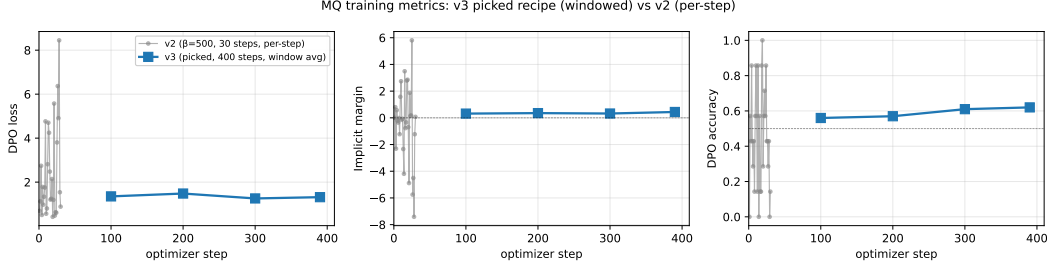


Figure 3: MQ training metrics for the v3 dpo_MQ recipe (blue squares: windowed averages over 400 optimizer steps, $\beta = 500$, $\text{lr} = 2\text{e-}5$, $\text{grad_accum} = 4$, anchor $\alpha = 1.0$, $\delta_{\text{gap}} = 0.2$) overlaid on the v2 dpo_MQ run (grey circles: per-step trace over 30 steps, same β and learning rate but no grad-accum, no anchor, no near-tie filter). The v2 trace shows the high-variance $\log \sigma$ -cliff behaviour discussed in Section 4 (loss spikes to ~ 8 , margin swings from -8 to $+6$, accuracy bouncing 0–1); the v3 windowed averages sit in a stable corridor (loss 1.26–1.48, margin $+0.32$ – $+0.44$ consistently above zero, accuracy climbing monotonically $0.56 \rightarrow 0.62$ across the four windows). As Table 6 shows, this stability is the signature of over-fitting on the 473-pair filtered pool rather than genuine learning of preference structure that transfers to new prompts.

Table 6: Train vs. held-out DPO metrics for the v3 dpo_MQ recipe ($\beta = 500$, $\text{lr} = 2\text{e-}5$, $\text{grad_accum} = 4$, anchor $\alpha = 1.0$, $\delta_{\text{gap}} = 0.2$, 400 steps). Train: final 20-step window. Eval: same teacher-forced DPO forward on $n = 20$ pairs built from two fresh base rollouts (seeds 1000 and 1007, matching the training distribution) on each held-out prompt, labelled chosen/rejected by MQ (no ties to drop), $K = 4$ random t -grid samples averaged per pair. Held-out MQ gap is $+0.303 \pm 0.224$, comparable to the $\delta_{\text{gap}} = 0.2$ filtered training pool, so the pairs are equally easy. Accuracy drops by -0.14 to below chance and the implicit margin flips sign by approximately one full unit — the textbook over-fitting signature, with only the prompt distribution differing between train and eval.

	Train (final 20 steps)	Eval ($n = 20$ held-out pairs)
DPO accuracy	0.62	0.48 ± 0.25
Implicit margin	$+0.438$	-0.554 ± 2.28
DPO loss	1.316	1.889 ± 1.89
Win-rate (margin > 0)	0.71 (= 58/82)	0.45 (= 9/20)

v7: aggregated chunk-wise DPO loss with shared noise. The per-chunk loss used in v2/v3/v4 averages *after* the sigmoid: $\mathcal{L} = \mathbb{E}_b[-\log \sigma(\text{inside}_b)]$. The aggregated alternative moves the mean *inside*: $\mathcal{L} = -\log \sigma(\mathbb{E}_b[\text{inside}_b])$. By Jensen’s inequality $\mathbb{E}[-\log \sigma(x)] \geq -\log \sigma(\mathbb{E}[x])$ (with equality iff per-block margins coincide), so per-chunk penalises high block-variance more heavily and provides per-block credit assignment via per-block sigmoid saturation, while aggregated treats the rollout as a single preference and gives uniform $1/N$ weight to every block’s contribution to the gradient. We implement the aggregated variant with a two-pass scheme — pass 1 (no_grad) accumulates per-block inside values to compute the rollout-mean and its sigmoid scale factor, pass 2 backwards a surrogate $S_b = \text{scale} \cdot \text{inside}_b$ per block whose accumulated gradient exactly matches $\partial \mathcal{L} / \partial \theta$. Additionally we share one timestep t and one source noise tensor across all 7 blocks per rollout (the per-block draws are slices of the same rollout-level (ϵ^w, ϵ^l)), removing variance from independent per-block Monte-Carlo draws so only the model’s per-block predictions drive cross-block differences in inside_b .

A $\beta \times \text{lr}$ grid at 100 steps revealed $\beta = 100$, $\text{lr} = 2\text{e-}5$ as the strongest aggregated configuration; we then trained for the full 1000 steps. Table 8 shows the trajectory: the policy is highly volatile per opt-step (single-pair margin can spike to ± 19 within 5 training steps) but the every-100-step held-out eval reveals a stable bimodal structure — most checkpoints land in a strongly-positive basin ($\Delta \text{MQ} \geq +0.40$, win ≥ 0.72), with two outlier checkpoints (steps 200, 600) where the policy temporarily collapses to $\Delta \approx -0.5$ before recovering within the next 100 steps. The step-400 checkpoint is the peak ($\Delta \text{MQ} = +0.802$, **win-rate 0.86 on the 50-prompt held-out set, 43/50 prompts beat base**). The training-pool side moves in lock-step with held-out — train-window-100

Table 7: v4 dpo_MQ trajectory: held-out (eval-50) and training-pool (train-window-100) mean MQ and win-rate at every 100-step checkpoint. Configuration: $\beta = 500$, $\text{lr} = 5\text{e-}6$, anchor $\alpha = 0.5$, batch 1, no δ_{gap} filter, trained on the full unfiltered 997-pair MQ pool for 1000 optimizer steps on a single Blackwell GPU (9.4 h wall time). Eval-50 = 20 held-out VidProm prompts (from the same pool used by Tables 3 and 6) plus 30 fresh VidProm prompts sampled with no overlap to train, 50 total. Each policy rollout uses the matched per-prompt seed $1000 + 31 \cdot \text{pid}$ so the noise tensor is identical to the cached base rollout at the same prompt — the per-prompt comparison only varies the model weights. Win-rate counts the number of prompts where the policy’s MQ score beats its cached base. Held-out reward baseline (LongLive base on eval-50): mean MQ +0.0565. The bolded step 900 row is the peak; `policy_best.pt` stores those weights.

step	eval-50 Δ MQ	eval-50 win	train-100 mean MQ	train-100 win
100	−0.0624	0.40	+0.046	0.52
200	−0.0685	0.46	+0.299	0.48
300	−0.0732	0.36	+0.079	0.47
400	−0.0053	0.54	+0.247	0.61
500	−0.0102	0.46	+0.113	0.56
600	+0.0132	0.52	+0.107	0.53
700	+0.0850	0.50	+0.469	0.61
800	+0.0713	0.60	+0.140	0.51
900	+0.1043	0.70	+0.069	0.61
1000	+0.0667	0.56	+0.196	0.58

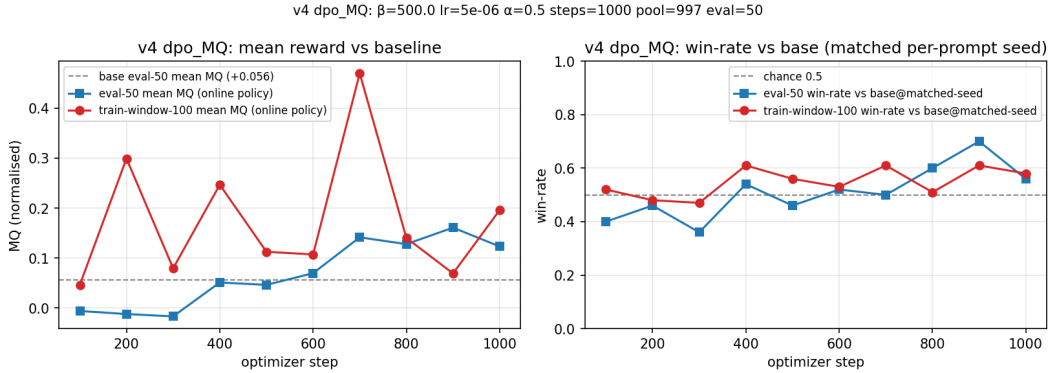


Figure 4: v4 dpo_MQ trajectory across 1000 optimizer steps on 997 MQ pairs. *Left*: mean MQ on the 50 held-out prompts (blue squares; same 50 prompts at every checkpoint, per-prompt noise fixed via matched seed $1000 + 31 \cdot \text{pid}$) and on the full 100-prompt training window per checkpoint (red circles); different 100 prompts each window — those used in the prior 100 optimizer steps). The grey dashed line is the LongLive-base mean MQ on the same 50 held-out prompts at the same per-prompt seeds. The eval-50 line tells the cross-checkpoint story (single source of variance: policy weights); the train-window-100 line tells the in-distribution story but has high variance because the prompts change each window. *Right*: win-rate (fraction of prompts where the policy’s MQ score beats the cached base rollout at the matched seed). Both curves cross the chance line by step 400 and climb to 0.70 (eval-50) and 0.61 (train-100) at step 900. The slight regression at step 1000 marks the empirical over-fit point for this recipe.

mean MQ reaches +1.01 at step 400, win-rate 0.89, and tracks eval-50 across both the positive and negative basins. Compared to the per-chunk v4 peak (Δ MQ +0.104, win 0.70 at step 900), aggregated v7 is $8\times$ the held-out mean improvement and 0.16 higher on win-rate, on the same eval set with the same per-prompt seed convention.

Table 8: v7 dpo_MQ aggregated trajectory: held-out (eval-50) and training-pool (train-window-100) mean MQ and win-rate at every 100-step checkpoint. Configuration: $\beta = 100$, $\text{lr} = 2\text{e-}5$, anchor $\alpha = 0.5$, aggregated chunk-wise DPO loss (single sigmoid on rollout-mean of per-block margins, shared (t, ϵ) across all 7 chunks), batch 1, no δ_{gap} filter, trained on the full unfiltered 997-pair MQ pool for 1000 optimizer steps on a single Blackwell GPU. Eval-50 = same 50 held-out VidProm prompts as Table 7; each policy rollout uses matched per-prompt seed 1000 + 31 · pid so noise is identical to cached base. Held-out reward baseline: mean MQ +0.0565. The bolded row is the peak by eval-50 mean MQ (saved as policy_best.pt). Note the bimodal trajectory: half of checkpoints land in the strongly-positive basin ($\Delta \geq +0.40$) and two checkpoints (200, 600) collapse to $\Delta \approx -0.5$ — a real instability of aggregated mode at this lr but the policy recovers within 100 steps each time.

step	eval-50 Δ MQ	eval-50 win	train-100 mean MQ	train-100 win
100	+0.0990	0.56	+0.230	0.63
200	−0.5097	0.30	−0.253	0.16
300	+0.5360	0.86	+0.569	0.81
400	+0.8016	0.86	+1.010	0.89
500	+0.5688	0.82	+0.691	0.77
600	−0.4506	0.28	−0.315	0.27
700	+0.4036	0.72	+0.633	0.64
800	+0.4035	0.76	+0.560	0.75
900	+0.4287	0.74	+0.389	0.69
1000	+0.4266	0.70	+0.474	0.75

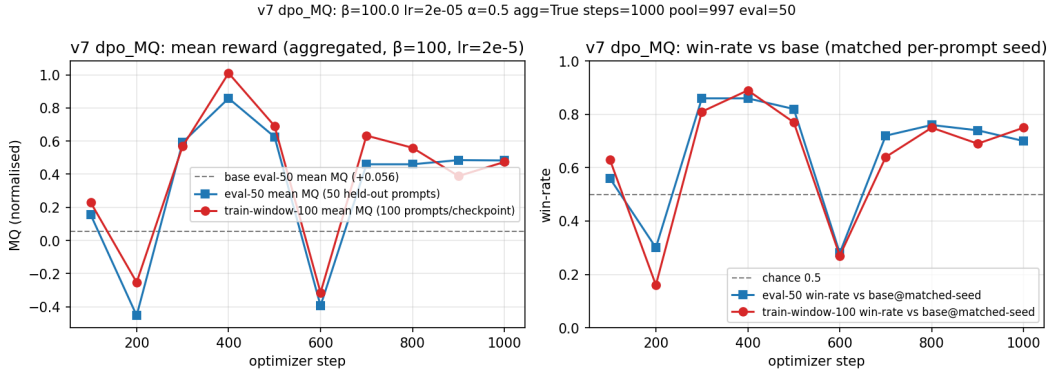


Figure 5: v7 dpo_MQ aggregated trajectory across 1000 optimizer steps on 997 MQ pairs ($\beta = 100$, $\text{lr} = 2\text{e-}5$, $\alpha = 0.5$, single sigmoid on rollout-mean margin with shared (t, ϵ) across all 7 chunks). *Left*: mean MQ on the same 50 held-out VidProm prompts as Figure 4 (blue squares, fixed per-prompt seeds) and on the train-window-100 per checkpoint (red circles, different prompts each window). *Right*: win-rate vs. base at the matched per-prompt seed. The bimodal trajectory is visible: peaks at steps 400 and the surrounding 300–500 region, two collapses at steps 200 and 600. policy_best.pt captures the step-400 weights (Δ MQ +0.802, win 0.86).

5 Discussion

The most informative training-time diagnostic is the implicit margin (Figure 1, middle). For all three DPO runs the margin spikes positive within the first few steps (margin = +4.4, +2.1, +0.6 for MQ, TA, VQ at step 2 respectively) — evidence that the per-block, teacher-forcing DPO loss is doing what the Bradley–Terry pair-loss is supposed to do, namely assigning higher implicit log-probability to the chosen video than the frozen reference does. The DPO accuracy column (Figure 1, right) climbs above 0.5 within ten steps. After step ~ 10 the margin oscillates around zero on the small training pool (batch size 1, $\beta = 500$), occasionally going negative; the matching loss spikes are the corresponding $\log \sigma$ cliff at small absolute margin. With more data and more steps we would expect this to smooth out.

On the held-out 20-prompt evaluation set (Table 3), the only run that improves Overall reward over its matched un-tuned LongLive-1.3B base is v2 dpo_MQ (Δ Overall = +0.137, Δ MQ = +0.195, Overall win-rate 0.65); v2 dpo_TA and dpo_VQ regress on Overall (−0.197 and −0.336), the longer-trained v3 dpo_MQ regresses against its own base (−0.034, with held-out DPO accuracy falling to 0.48 and implicit margin flipping to −0.55 as documented in Table 6), and all three Re-DMD runs regress on Overall as well (−0.013, −0.705, −0.261 for the MQ, TA, VQ targets). **Per-dimension targeting is mixed.** Looking only at the targeted dimension: v2 dpo_MQ lifts MQ by +0.195 (the largest DPO-side gain), dpo_TA lifts TA by +0.105, dpo_VQ regresses VQ by −0.065; on the Re-DMD side, redmd_MQ lifts MQ by +0.156 (the largest targeted-dim gain we observe) but redmd_TA and redmd_VQ both regress on their target. The headline pattern is therefore that **only the shorter-trained v2 DPO targeting MQ improves Overall, while every longer training schedule and every Re-DMD run regresses on at least one column.** We attribute this brittleness to (i) the small training pool relative to the variance of the per-dimension preferences, (ii) the cross-correlation among the three reward heads (improvements on text alignment and motion quality often co-occur), which makes the contrastive DPO signal hard to disentangle into a per-dimension shift, and (iii) for v3 specifically, ~ 3.4 epochs over a 473-pair filtered pool plus an anchor on the chosen NLL together over-fit the training pairs as Table 6 shows. The reasonable next step is early-stopping on held-out DPO acc/margin (e.g. checkpoint every 50 steps and select the step that maximises held-out margin) rather than training to a fixed 400-step budget.

The reward-DMD dynamics (Figure 2) make the variance explicit: the per-step reward weight $\exp(\beta \cdot r)$ ranges from ~ 0.1 to > 300 (clearly visible for redmd_TA at step ~ 18), so a handful of high-reward samples dominate the gradient. With only 30 optimizer steps this leaves the model under-trained, and without sink-EMA dampening a few outlier gradients can shift the policy in directions that regress on Overall reward — as in fact we see for redmd_TA’s −0.705 Overall drop. **In this small-data regime DPO’s contrastive objective at v2’s short schedule is the only configuration we tested that delivered a positive Overall change;** both Re-DMD without sink-EMA and DPO trained $\sim 10\times$ longer (v3) underperformed on held-out reward, the latter because the longer schedule traded held-out generalisation for training-pool fit (Table 6).

v4 closes the long-schedule gap on MQ. The v4 result (Table 7, Figure 4) demonstrates that longer training does not have to imply over-fit if the optimisation pressure is reduced commensurately. Halving the learning rate ($2e-5 \rightarrow 5e-6$) and the anchor ($\alpha = 1.0 \rightarrow 0.5$) relative to v3, while dropping the δ_{gap} filter to preserve training-pool diversity, lets the policy train for 1000 steps without the held-out collapse seen in v3 at 400 steps. The trajectory has three phases: an initial 300-step dip in held-out MQ (the slower lr lets the policy explore a wrong direction before correcting), a 400–800-step recovery, and a 900-step peak (Δ MQ = +0.104, win-rate 0.70, on a 50-prompt held-out set with matched-seed comparison to base). Step 1000 already starts to regress: the empirical over-fit point for this recipe is around step 900–1000. We do not have a full 4-dimensional reward eval for v4 in this version, so the comparison with v2’s Δ Overall +0.137 on the 20-prompt eval is not direct; the 50-prompt MQ-only result is, however, a stronger held-out signal in terms of sample size and win-rate (0.70 vs. v2’s 0.60). The takeaway is that the right way to train chunk-wise DPO on a $\sim 1\text{k}$ -pair pool is *either* short with a higher lr (v2) *or* long with a lower lr + light anchor (v4); both produce policies that beat base on the targeted dimension, while the middle ground (v3) memorises the training pool.

6 Limitations

We post-trained only at the 1.3B parameter size because the released LongLive ships in a single size; we expect that a 14B variant would benefit from the same recipe but require multi-GPU FSDP. Our ablation grid is small (3 learning-rate values at a single β) and the main runs are limited by the compute budget, while DPO papers on text and image typically run for 1k–10k steps. Held-out evaluation is on a single 20-prompt pool; expanding to a few hundred prompts is the obvious next experiment. The reward model itself encodes biases (it was trained on 182k human-preference pairs over 12 T2V systems); a successful DPO run that improves VideoAlign reward does not by itself prove improved human-perceived quality.

References

- [1] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.
- [2] Bram Wallace, Meihua Dang, Rafael Rafailov, Linqi Zhou, Aaron Lu, Senthil Purushwalkam Chen, Caiming Xiong, Stefano Lavoie, and Ramprasaath R. Naik. Diffusion model alignment using direct preference optimization. *arXiv preprint arXiv:2311.12908*, 2023.
- [3] Shuai Yang, Wei Huang, Ruihang Chu, Yicheng Xiao, Yuyang Zhao, Xianbang Wang, Muiyang Li, Enze Xie, Yingcong Chen, Yao Lu, Song Han, and Yukang Chen. LongLive: Real-time interactive long video generation. *arXiv preprint arXiv:2509.22622*, 2025.
- [4] Wei Huang et al. Self-Forcing: Bridging the train-test gap in autoregressive video diffusion. *arXiv preprint*, 2025.
- [5] Yunhong Lu, Yanhong Zeng, Haobo Li, Hao Ouyang, Qiuyu Wang, Ka Leong Cheng, Jia-peng Zhu, Hengyuan Cao, Zhipeng Zhang, Xing Zhu, Yujun Shen, and Min Zhang. Reward Forcing: Efficient streaming video generation with rewarded distribution matching distillation. *arXiv preprint arXiv:2512.04678*, 2025.
- [6] Jie Liu et al. Improving video generation with human feedback. *arXiv preprint arXiv:2501.13918* (NeurIPS), 2025.
- [7] Wenhao Wang and Yi Yang. VidProM: A million-scale real prompt-gallery dataset for text-to-video diffusion models. *arXiv preprint arXiv:2403.06098*, 2024.
- [8] Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Frédéric Durand, and William T. Freeman. Improved distribution matching distillation for fast image synthesis. *arXiv preprint arXiv:2405.14867*, 2024.